

2026 CodeEngn Conference

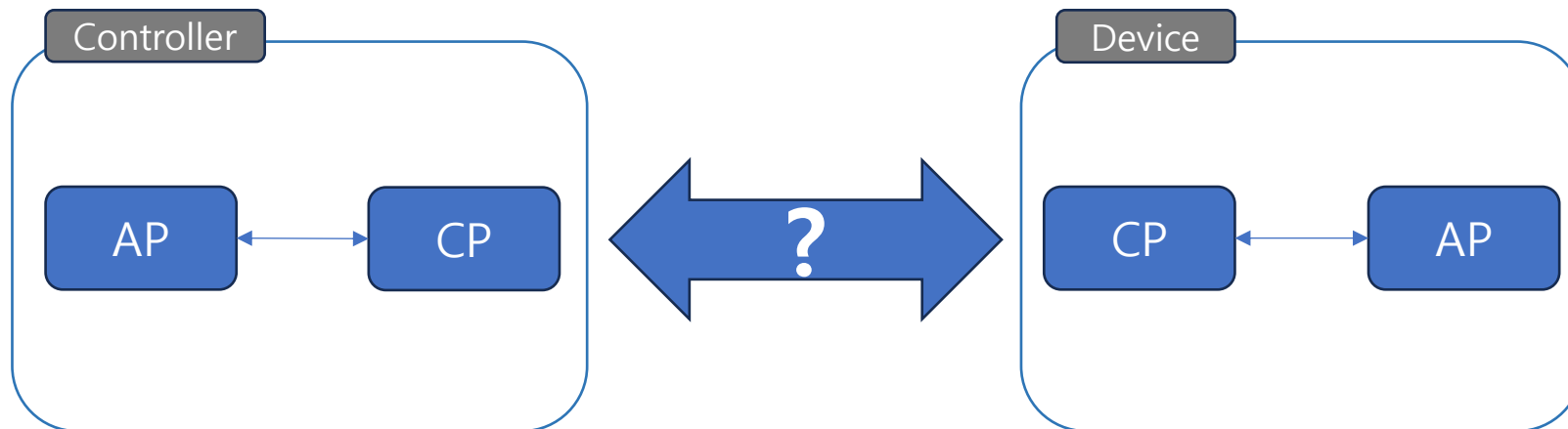
LLM아 Stripped Binary에서 함수 찾아줘

임베디드 펌웨어 정적 분석 기반 통신 프로토콜 및 Task 식별 방법론 연구

발표자: 연동현 (26.07.25)

상용 임베디드 장비 리버싱

- 분석 목표
 - 통신 프로토콜 규명



희소 아키텍처 AVR32 위성 펌웨어 분석을 위한 생성형 AI 기반 함수 이름 자동 추론 기법

Generative AI-Based Function Name Recovery for Sparse AVR32 Satellite Firmware Architectures

발표자: 연동현 (경희대학교 융합보안대학원 컴퓨터공학과)

TABLE OF CONTENTS

01 | 연구 배경

02 | 연구 목표

03 | 제안 기법: LEAF 프레임워크

04 | 실험 및 분석

05 | 관련 연구

06 | 결론

민간 기업이 이끄는 'New Space' 시대 도래

✓ 민간 주도 우주 산업의 부상과 위성 보안 이슈

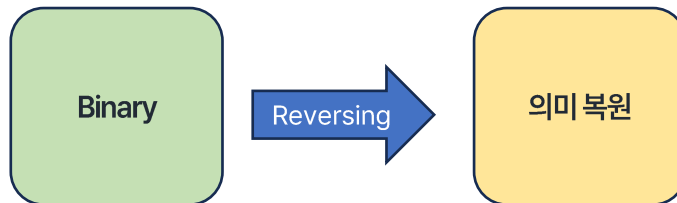
- **New Space 확산**
 - 정부·군 중심 우주 개발에서 민간 주도 상업 우주 산업으로 전환
- **COTS 기반 상용 OBC 플랫폼 도입 증가**
 - CubeSat 시장 성장과 함께 GomSpace NanoMind 등 상용 OBC 플랫폼 활용 확대
- **OBC 펌웨어가 핵심 제어 로직을 담당**
 - OBC 펌웨어가 통신·명령 처리·임무 제어의 핵심 제어 로직을 수행
- **취약점 발생 시 임무 실패 및 위성 운용 리스크로 연결**
 - 궤도상 원격 운용 환경에서 펌웨어 취약점은 통신 두절·임무 실패로 직결 가능



위성 펌웨어 분석을 위한 바이너리 리버스 엔지니어링

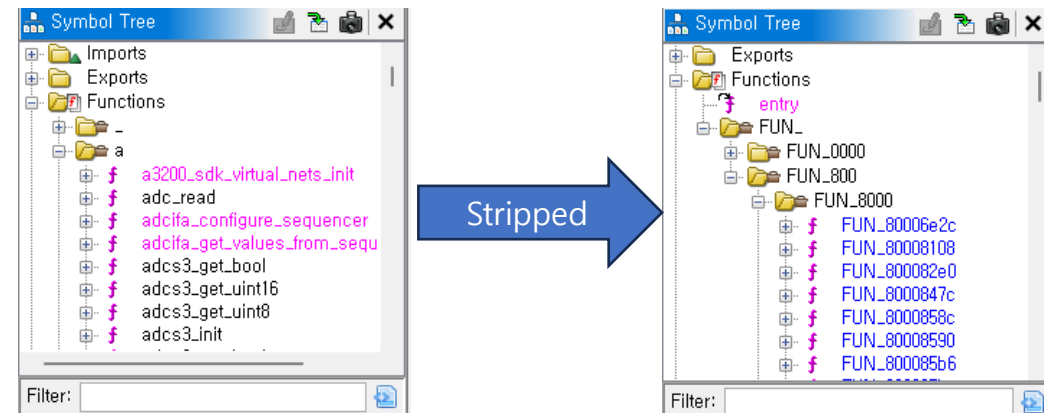
- ✓ **바이너리 리버스 엔지니어링**(Binary Reverse Engineering) : 실행 파일을 역으로 해석해 내부 동작과 의미를 복원하는 기술
- ✓ **Stripped Binary**는 디버깅 심볼이 제거되어 함수명과 변수명 등 의미 단서가 사라지므로, 함수의 역할을 파악하기 어려움

소스 코드 부재 환경



- 상용 펌웨어는 IP 보호를 위해 **디버깅 심볼이 제거된 Stripped Binary**로 배포
- 함수명, 변수명 등 **의미론적 정보가 소실**되어 어셈블리 수준 해석이 불가피
- 보안성 검증을 위해 **바이너리 리버스**하는 것이 사실상 유일한 접근 수단

Stripped Binary 리버스 엔지니어링



✓ 위성 펌웨어 분석의 어려움

하드웨어 의존성

- 독자 규격 SoC를 채택하여, 공개된 데이터시트나 레퍼런스 문서가 거의 존재하지 않음
- MMIO 레지스터와 주변장치의 동작을 문서가 아닌 코드 패턴으로 추정해야 함
- 메모리 맵과 I/O 경로를 직접 복원해야 함
-> 초기 분석 비용 증가

동적 계측 제한적

- 대상 펌웨어는 RTOS 기반 바이너리로 동작하여, Frida 등 동적 후킹, 계측 도구를 적용할 수 없음
- 실행 중 함수 호출과 인자를 관찰할 수 없어 정적 분석에 전적으로 의존해야 함
- 동작을 확인하려면 자체 디버깅 환경을 직접 구축해야 해 분석 진입 장벽이 큼

희소 아키텍처

- AVR32 등 희소 아키텍처에 대해서는, 공개 코드, 분석 사례가 극히 제한적
- 표준 분석 도구, 학습 데이터셋이 부재하여 기존 자동화 분석 기법 적용이 어려움
- IDA Pro는 AVR32 공식 지원 종료, 실질 분석은 Ghidra에 전적으로 의존

Mainstream
Architectures
(x86, ARM)

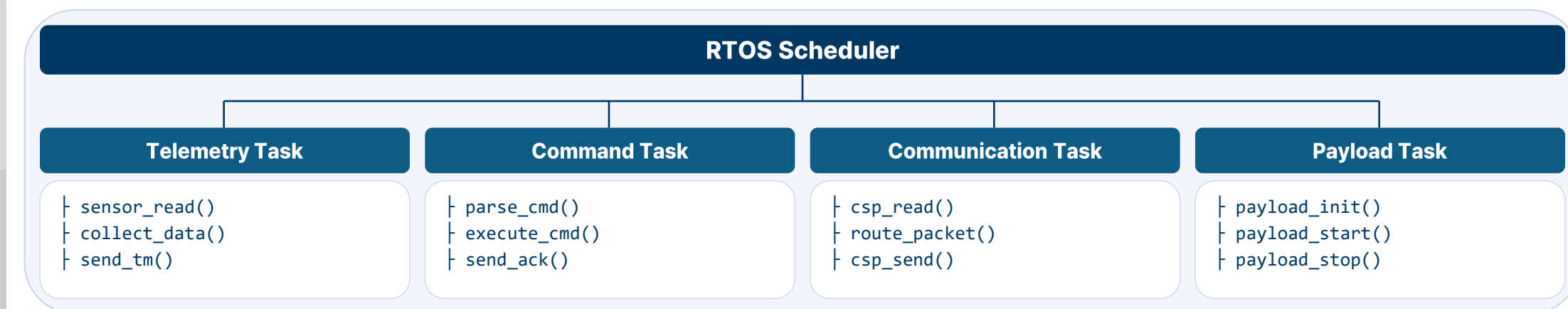
Sparse
Architectures
(AVR32)

분석난이도

분석의 첫 관문: Task의 이해

- ✓ RTOS 기반 펌웨어는 **다수의 Task**가 병렬적으로 동작
- ✓ 따라서 보안 분석은 **"어떤 Task가 어떤 기능을 담당하는가"**를 먼저 파악하는 것에서 출발

수백 ~ 수천의 Stripped 함수로 인해 Task 분석 난이도 높음



연구 목표

Task 및 함수 역할 후보를 자동 추론하여, 분석자가 검토할 범위를 줄이는 AI-assisted 분석 프레임워크 구축

LEAF는 분석가를 대체하는 도구가 아니라, stripped firmware에서 함수-Task 의미 복원의 출발점을 제공하는 분석 보조 프레임워크

설계 요구사항 도출

✓ 기존 접근의 한계

- 1 AVR32 공개 코드, 사례 부족으로 학습 데이터 확보 어려움
- 2 단일 함수 분석만으로는 Task 단위 역할 추론이 어려움
- 3 LLM 단독 생성은 결과 검증 어려움



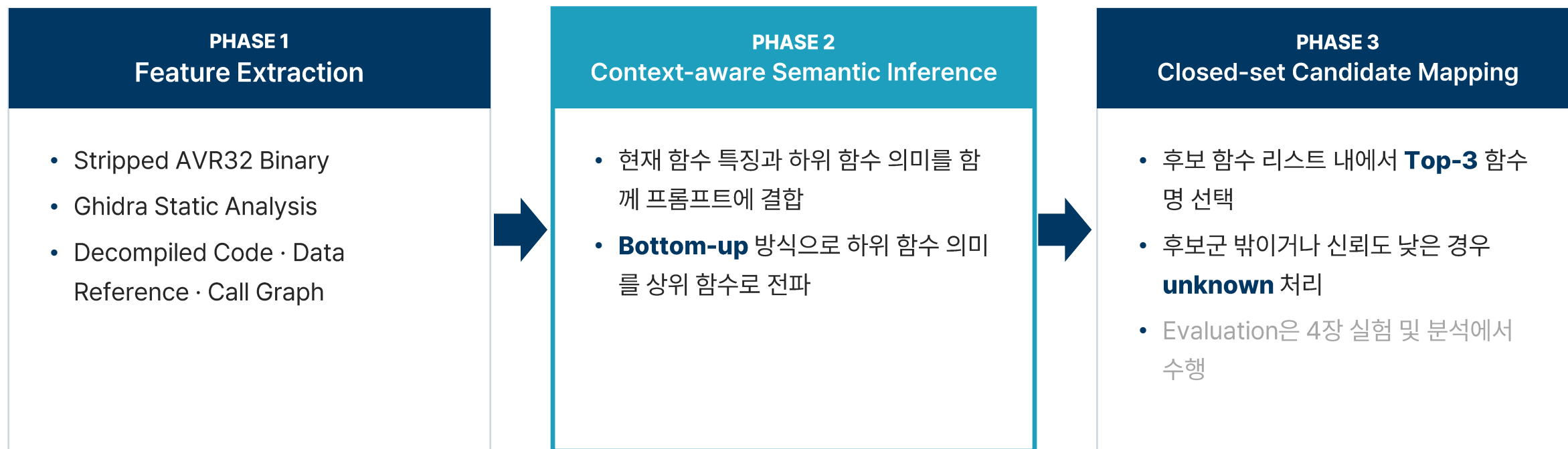
✓ 설계 요구 사항

AVR32와 같은 희소 ISA에서도 동작하도록 특정 아키텍처 학습 데이터에 의존하지 않는 분석 구조 필요

단일 함수 분석의 한계를 보완하기 위한 추가적인 정보 활용 필요

LLM 추론 결과의 신뢰성을 확보하기 위한 방법 필요

LEAF: 함수 이름 추론을 위한 3-Phase 파이프라인



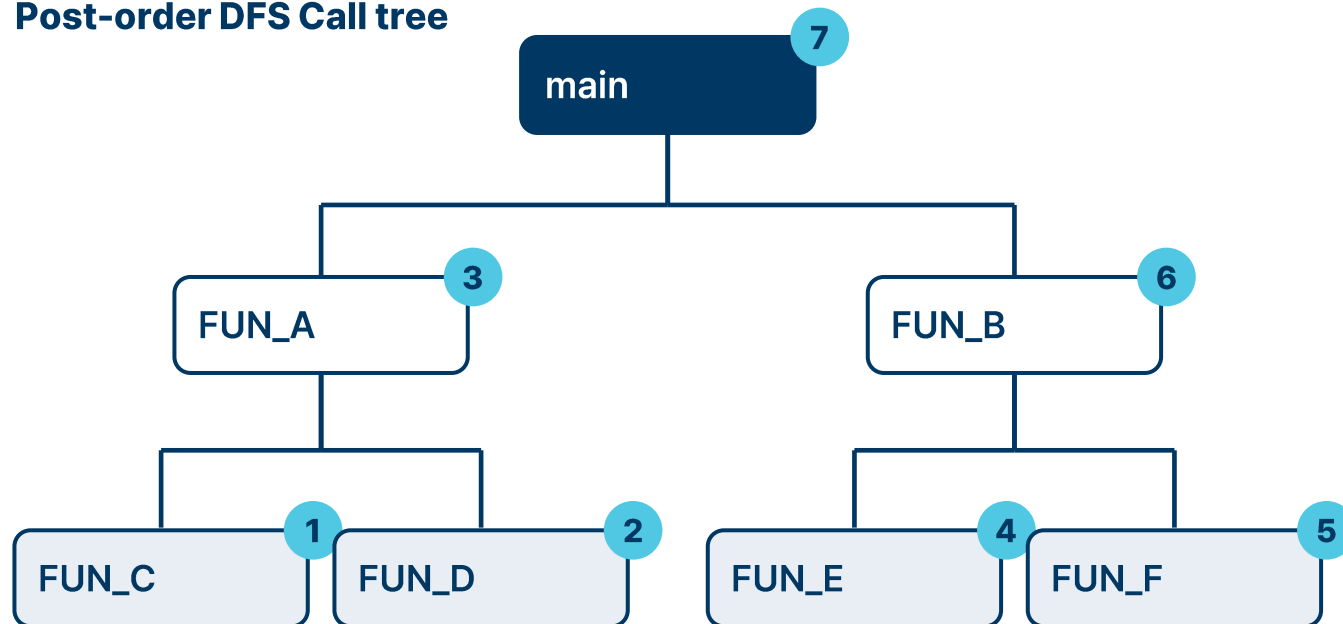
LEAF는 Ghidra 의사코드와 호출 그래프를 기반으로 하위 함수 의미를 상위 함수로 전파하고, 후보군 내에서 함수명을 제한적으로 추론한다.

03 제안 기법: LEAF 프레임워크

✓ 핵심 원리: Post-order DFS 기반 Context Propagation

호출 그래프의 LEAF 함수부터 Root/Task 방향으로 **Post-order DFS 순회**로 문맥 기반 의미 추론 수행
 대상 함수 분석 전 **Callee 노드를 우선 탐색**하여, 하위 함수 의미가 준비된 상태에서 상위 함수를 추론
 분석 완료된 하위 함수의 **예측 함수명·요약·신뢰도를 캐시에 저장**하고, 상위 함수 프롬프트에서 의미 단서로 재사용

Post-order DFS Call tree



Context Propagation
 Bottom-up 의미 복원

✓ Phase 1. Feature Extraction

각 함수 노드에서 **의사코드**, **데이터 참조**, **호출 관계** 세 가지를 단서로 추출하여 LLM 입력으로 사용

단순히 표면적 코드 패턴만 보는 것이 아니라, **함수가 다루는 데이터와 주변 호출 문맥**을 함께 묶어 의미 추론의 근거를 확보

01

Decompiled Code

Ghidra 의사코드 기반 제어 흐름과 연산 구조

분기, 반복, 메모리 접근 패턴을 텍스트 단서로 확보

02

Data Reference

문자열, 상수, 테이블, 레지스터 참조 등 의미 단서

I/O 매핑, 상수 사전으로 함수의 도메인을 식별

03

Caller-Callee Graph

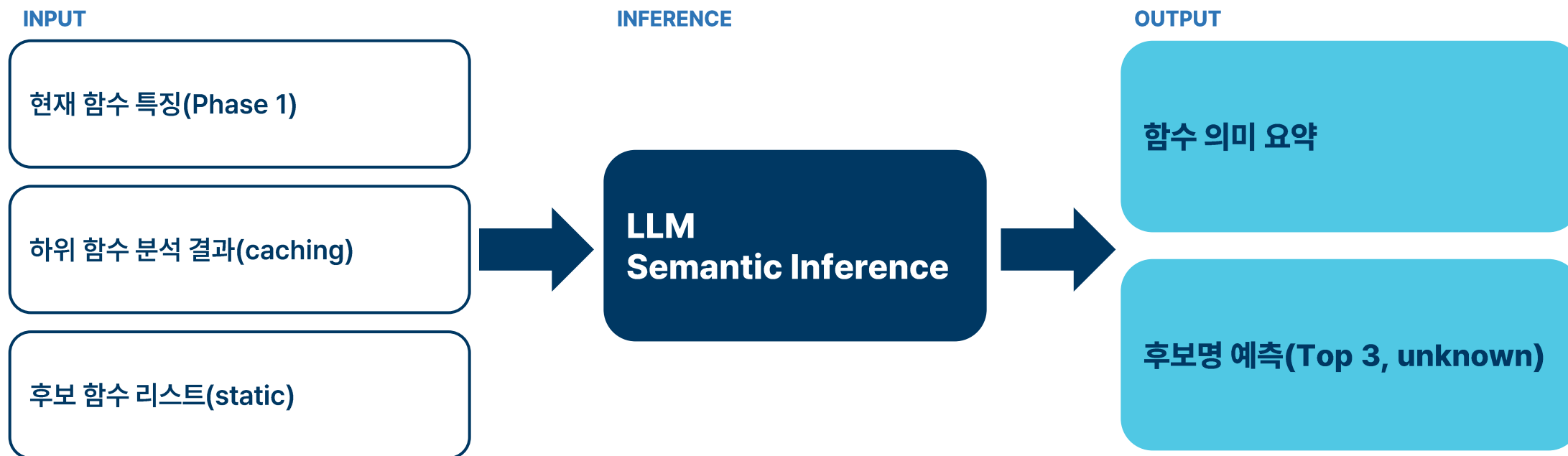
하위 함수 및 호출 관계 기반 문맥 정보

이미 추론된 하위 함수 의미를 상위 문맥에 연결

03 제안 기법: LEAF 프레임워크

✓ Phase 2 & 3. LEAF에서의 LLM 추론 설계

- ① **Zero-shot Inference** AVR32 라벨 데이터 부족 환경에서 파인튜닝 없이 Ghidra 의사코드의 기능 단서를 추론
- ② **Context Propagation** 이미 분석된 하위 함수 의미를 상위 함수 프롬프트에 삽입하여 Task 문맥 반영
- ③ **Closed-set Output** 후보 함수 리스트 내에서 Top-3 선택, 후보군 밖 결과는 unknown으로 처리



LEAF는 생성형 AI의 자유 생성을 후보군 기반 판단 문제로 제한하여, 환각을 줄이고 검증 가능한 출력을 생성

✓ Phase 2 & 3. LEAF에서의 LLM 추론 설계 (cont'd)

대상 함수 분석 전 **Callee**노드를 우선 탐색하여, 하위 함수 의미가 준비된 상태에서 **상위 함수를 추론**

분석 완료된 **하위 함수의 함수명, 요약, 신뢰도를 캐시에 저장**, 하위 Callee의 의미 흐름을 결합해 **상위 함수의 의미 맥락을 보정**

1. Child Analysis Results (Cache)

```
Child 1 (FUN_8003af08):  
{  
  "predictions": [  
    {"name": "mbedtls_zeroize",  
     "confidence": "High"}, ...  
  ]  
}
```

```
Child 2 (FUN_80009678):  
{  
  "predictions": [  
    {"name": "memcpy",  
     "confidence": "Medium"}, ...  
  ]  
}
```

2. Python Formatting Logic

```
.build_prompt()  
  
Extract top-1 predictions  
and format context string  
  
f"{addr} -> {name} ({confidence})"
```

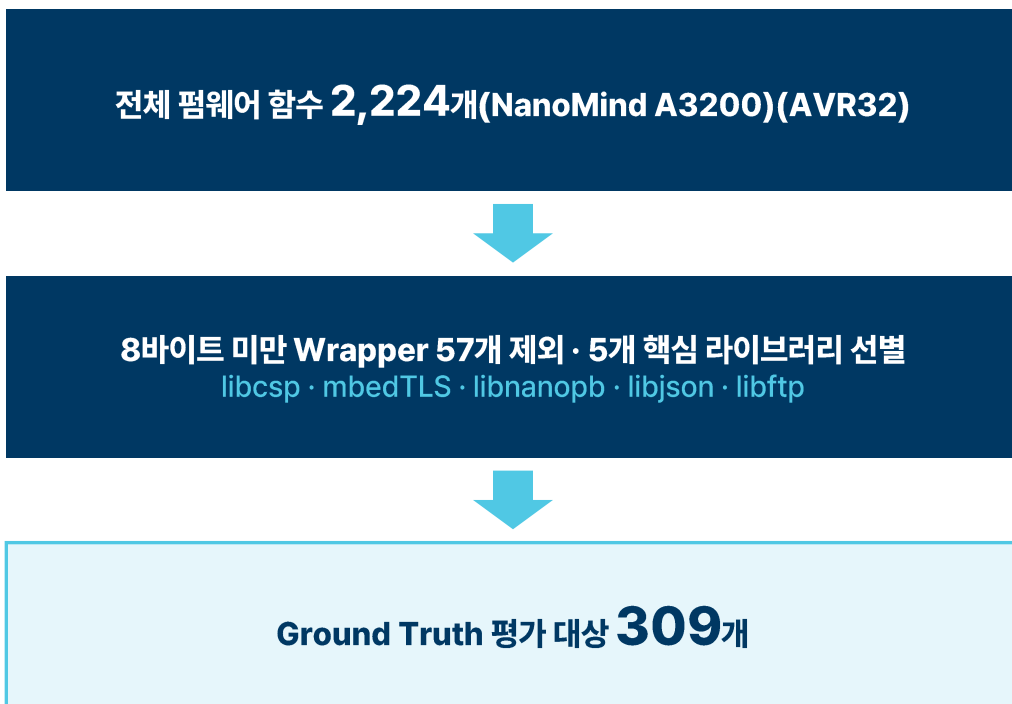
3. Injected Prompt Context

```
**Target Function:**  
- Name: FUN_8003904c  
- Address: 0x8003904C  
- Size: 3164 bytes  
  
**Children Context:**  
FUN_8003af08 -> mbedtls_zeroize (High)  
FUN_80009678 -> memcpy (Medium)  
} Injected children.ctx  
  
- String Refs: []  
- Constants: ["0xFF00"]  
  
**Decompiled Code:**  
if (param_2 < 2) { ... }
```

✓ 실험 환경 및 평가 데이터셋

AVR32 기반 **GomSpace NanoMind A3200 OBC 펌웨어**를 대상으로, Ground Truth가 확보된 **5개 핵심 라이브러리 309개 함수**를 평가

평가 대상 선정 흐름



평가 지표

지표	기준	설명
Top-1 Accuracy	Exact Match	예측 1순위와 정답 함수명의 완전 일치
Top-3 Accuracy	Top-3 Hit	상위 3개 후보 안에 정답 포함
Library ID	Library Match	예측 함수가 속한 라이브러리 단위 식별

Exact Match는 보수적 평가 기준, Library Match는 기능 범위를 좁히는 실용적 지표

※ 함수 경계 복원은 본 연구 범위 밖이며, LEAF는 복원된 함수 단위에 대한 의미 추론을 대상으로 함

✓ 전체 성능 결과

AVR32 Stripped Binary 환경에서 **Top-1 54.69%, Top-3 70.87%, Library Match 87.06%**의 의미 복원 성능을 달성

Top-1 Accuracy

54.7%

169 / 309

Top-3 Accuracy

70.9%

219 / 309

Library Match

87.1%

269 / 309

Hub Node Top-1

77.8%

상위 제어 함수 기준

LEAF는 완전한 함수명 확정 도구라기보다, 분석자가 검토할 후보군과 기능 범위를 효과적으로 줄이는 assistant로 유효 하위 함수 의미가 많이 누적되는 Task·상위 제어 함수(Hub Node)에서 LEAF의 효과가 가장 뚜렷함

✓ 라이브러리 특성에 따른 함수 식별 성능 차이

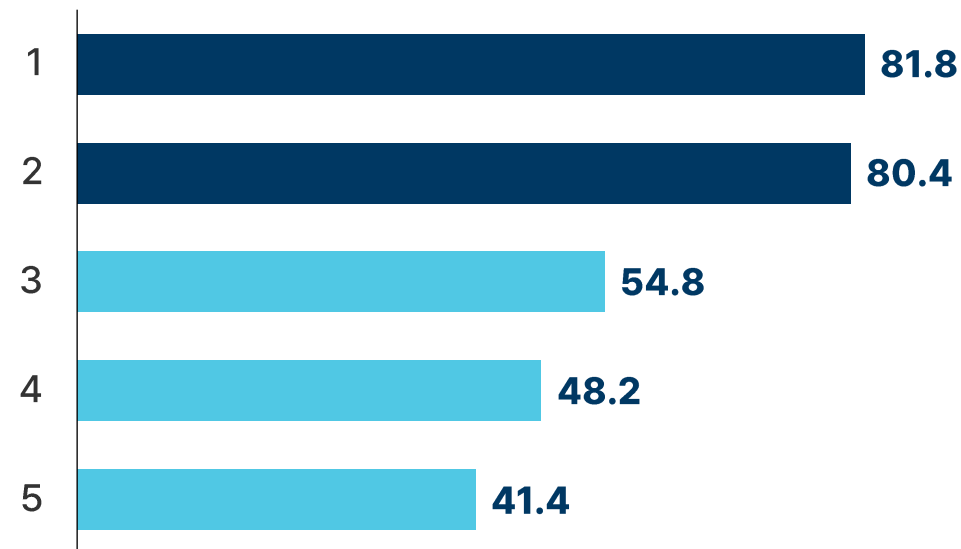
libnanopb와 **libjson**은 구조체 직렬화와 데이터 필드 접근 패턴이 명확해 **Top-1 80% 이상**의 높은 복원 성능

libcsp는 세부 함수명 복원은 중간 수준이나, **Library Match 96.0%**로 통신 계열 식별에 강함

mbedtls와 **libftp**는 반복 블록 연산과 cleanup·close 계열 함수의 구조 유사성으로 세부 함수명 구분이 어려움

Library	#Func	Top-1	Top-3	Lib ID
libjson	11	81.8%	81.8%	81.8%
libnanopb	46	80.4%	95.7%	100.0%
libcsp	126	54.8%	71.4%	96.0%
libftp	27	48.2%	66.7%	85.2%
mbedtls	99	41.4%	58.6%	70.7%

Top-1 Accuracy by Library



✓ 라이브러리 혼동 패턴 분석

LEAF는 대부분의 라이브러리를 올바른 기능군으로 식별(avg. 87%)

libnanopb

Lib ID **100.0%**

직렬화, 파싱 패턴이 명확해 가장 안정적으로 식별

libcsp

Recall **99.18%**

실제 통신 계열 함수 대부분을 올바르게 식별

mbedTLS

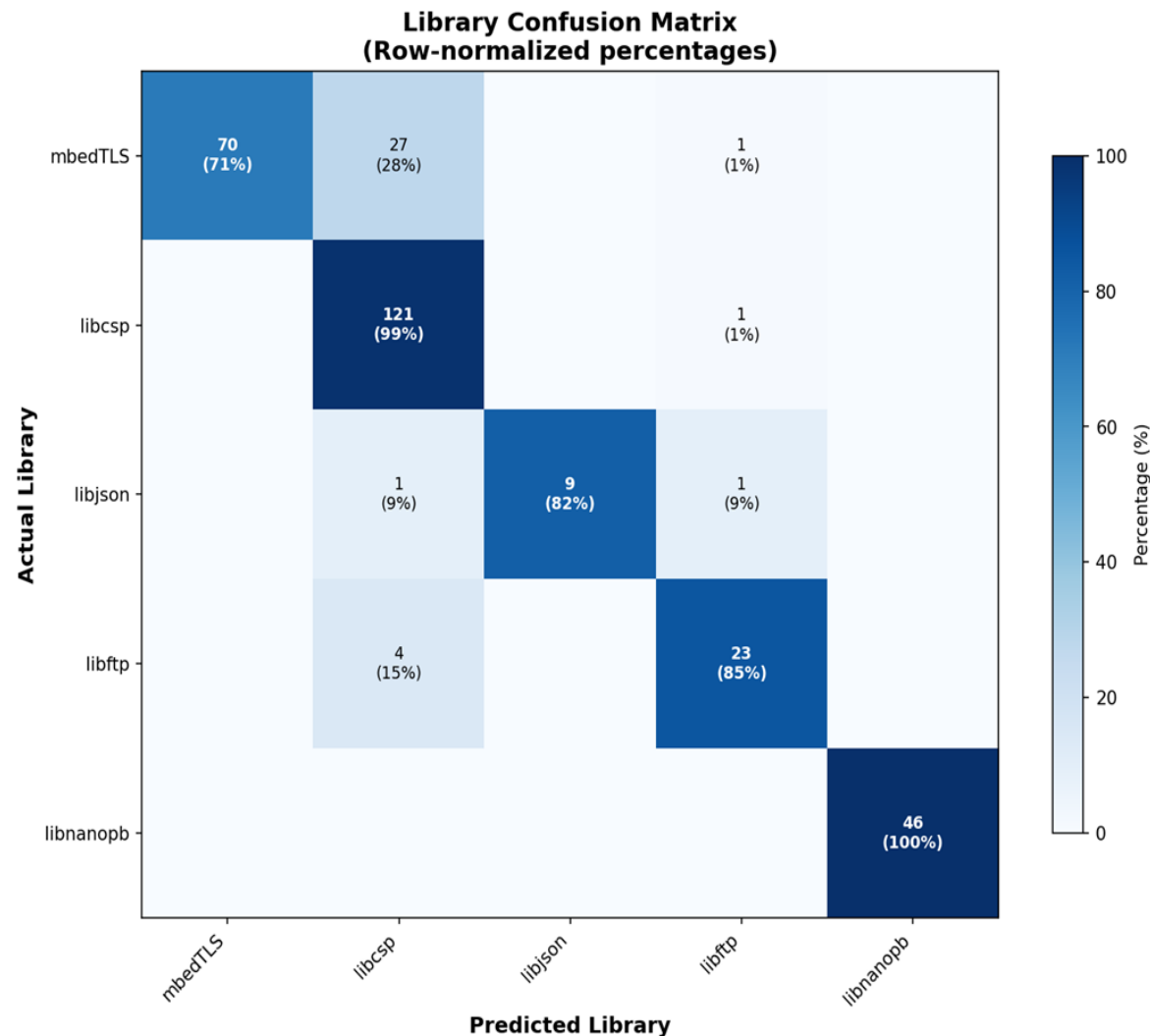
Precision **100.0%**

암호 함수로 예측한 경우 오탐 없이 정확히 분류

mbedTLS → libcsp

27 / 28 errors

cleanup/close 계열 구조 유사성으로 혼동 발생

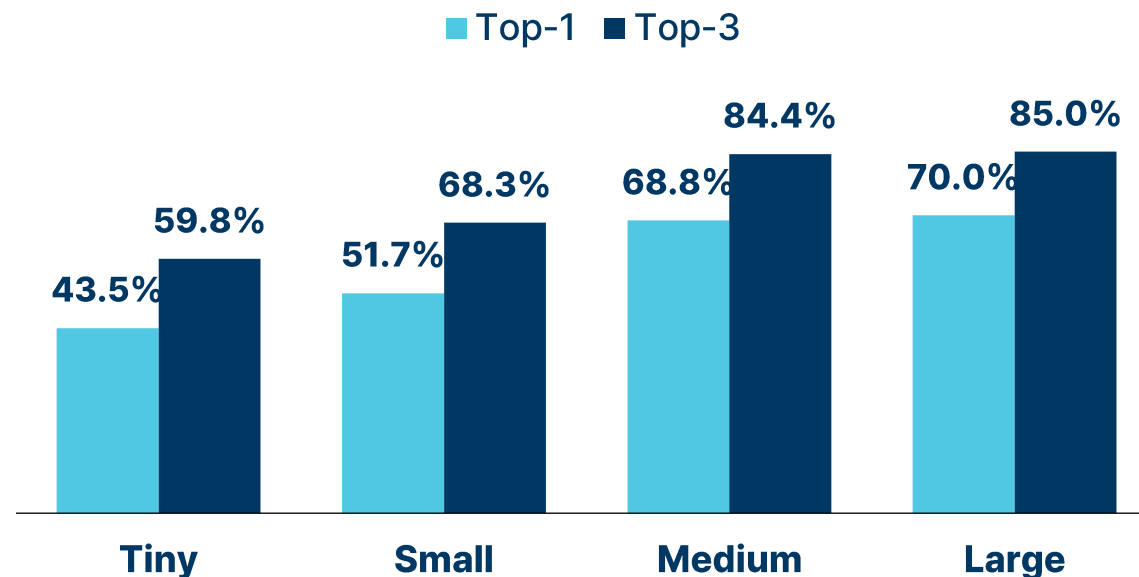


✓ 함수 크기가 성능에 미치는 영향

함수 크기가 증가할수록 제어 구조, 상수, 데이터 참조 등 **관찰 가능한 특징이 증가**하며 **함수명 복원 정확도가 향상**

Group	Range (Bytes)	Count	Top-1	Top-3
Tiny	8 - 32	92	43.5%	59.8%
Small	33 - 128	120	51.7%	68.3%
Medium	129 - 512	77	68.8%	84.4%
Large	513 +	20	70.0%	85.0%

함수 크기별 평균 정확도



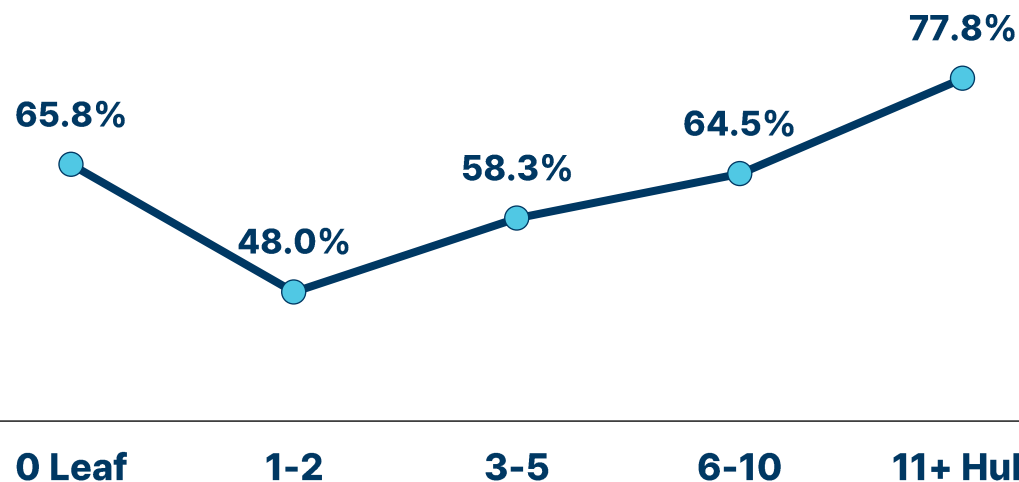
Function Evidence 설계는 코드, 데이터 특징이 충분한 함수에서 더 안정적인 추론 결과를 제공

✓ 호출 관계 기반 문맥 전파의 효과

하위 호출 수가 증가할수록 상위 함수 추론에 활용 가능한 문맥 정보가 증가하며, Hub Node에서 가장 높은 성능을 보임

Children	Count	Top-1	Top-3	Lib ID
0 (Leaf)	38	65.8%	71.1%	84.2%
1 - 2	171	48.0%	67.3%	86.5%
3 - 5	60	58.3%	71.7%	85.0%
6 - 10	31	64.5%	83.9%	93.5%
11+ (Hub)	9	77.8%	88.9%	100.0%

호출 자식 수에 따른 Top-1 (J-Curve)



다수의 하위 함수 의미가 결합될수록 상위 함수의 역할 추론에 필요한 문맥 정보가 증가함을 확인 → Context Propagation과 Bottom-up 구조 효과

정성적 분석: 성공 사례와 의미적 불일치 사례

- ✓ 성공 사례는 **고유 상수와 호출 패턴**이 명확한 함수에서 발생
- ✓ Top-1 불일치는 기능 이해 실패보다 추상화 계층 차이와 유사 API 구조에서 주로 발생

✓ 성공 사례: Top-1 정확 일치

TARGET FUNCTION

`mbedtls_aes_crypt_ecb()`

GROUND TRUTH `mbedTLS / mbedtls_aes_crypt_ecb`

MODEL PREDICTION **`mbedTLS / mbedtls_aes_crypt_ecb` ✓**

핵심 단서

- AES S-box 상수 테이블 직접 참조
- `mbedtls_internal_aes_encrypt` 호출
- 16바이트 블록 처리 패턴 일치
- **구조적 특징과 보안 문맥이 결합되어 정확한 함수명 복원**

✗ 의미적 불일치: Top-1 오분류

TARGET FUNCTION

`csp_close()`

GROUND TRUTH `libcsp / csp_close`

MODEL PREDICTION **`libcsp / csp_rdp_close` ✗**

혼동 원인

- 동일 라이브러리 내 연결 종료 계열 함수
- 호출 패턴, 자원 해제 시퀀스 유사
- RDP 세션 정리 단서가 과대 반영
- **추상화 수준 차이: 일반 종료 인터페이스를 구체적 RDP 종료로 예측**

기타 불일치 유형: `csp_rtable_set_internal` → `csp_rtable_set` (내부 구현 vs 외부 API 계층 차이) · `csp_accept` → `csp_read` (수신 처리 계열 명명 차이)

`mbedtls_sha1_init` → `mbedtls_ripemd160_init` (memset 기반 초기화 패턴으로 해시 계열 초기화 함수 구분 어려움)

Architecture-portable 설계 분석: x86-64 통제 실험 (Control Experiment)

- ✓ 동일 파이프라인을 **x86-64 통제 실험(Control Experiment)**으로 적용해 ISA 변화에도 유사한 추론 양상을 확인
- ✓ Top-1 오답의 상당수는 함수 이해 실패보다 **추상화 계층 차이와 구조적 유사성**에서 발생

Top-1 불일치의 의미 분석

1. 추상화 계층 차이

`csp_close` → `csp_rdp_close`

범용 종료 API와 RDP 세션 정리 함수 간 명명 계층 혼동

2. 구조적 구분 한계

`mbedtls_sha1_init` → `mbedtls_ripemd160_init`

동일한 초기화 패턴으로 알고리즘 고유 단서 부족

ISA-agnostic 분석 지원 검증

1. 추론 양상 보존

함수 크기, 하위 호출 수 성능 추이와 오답 양상이 유사하게 반복

2. 구조 기반 추론

opcode가 아닌 Ghidra 의사코드, 호출 그래프, 의미 구조를 활용

LEAF는 raw opcode가 아니라 Ghidra 의사코드와 호출 그래프를 입력으로 사용, **ISA 차이를 일부 추상화한 중간 표현 위에서 의미 추론을 수행**

✓ 실험 분석 종합: LEAF의 효과와 한계

LEAF는 AVR32 Stripped Binary 환경에서 분석자가 검토해야 할 **함수명 후보와 라이브러리 범위를 효과적으로 줄임**
초기 리버스 엔지니어링 과정을 지원하는 유용한 **분석 보조 프레임워크**로서의 가능성을 확인

기능군 식별

87.1% Library ID

함수가 속한 라이브러리, 기능 범위 식별에 효과적

문맥 전파 효과

77.8% Hub Node Top-1

하위 호출이 많은 Hub/Task 인접 함수에서 Bottom-up 문맥 전파 효과 확인

설계 효과 검증

Feature Extraction + Context Propagation

함수 크기와 하위 호출 수가 증가할수록 코드, 데이터 특징과 호출 문맥을 활용한 추론 효과가 확인됨

한계 확인

wrapper / cleanup-close / 유사 API

짧은 wrapper 함수, 자원 정리 계열, 유사한 API 구조에서는 세부 함수명 Exact Match에 한계가 존재

회소 아키텍처 환경에서 기존 기법의 구조적 한계

✓ 기존 함수 식별 기법은 **시그니처, 딥러닝, LLM 기반 접근**으로 구분되며, 각각 **참조 바이너리, 학습 데이터, 로컬 컨텍스트**에 의존

시그니처, 유사도 기반

FLIRT · BinDiff · Diaphora

한계 - 참조 바이너리, 시그니처 DB 의존

- AVR32 같은 회소 ISA는 공개 시그니처 DB가 제한적
- 제조사 커스텀 펌웨어는 기준 바이너리 확보가 어려움
- 컴파일러 최적화, 인라인으로 바이트 패턴이 쉽게 변형

학습 기반 함수 식별

NERO · Debin · Asm2Vec

한계 - 대규모 학습 데이터 의존

- 위성 펌웨어 도메인은 라벨링 데이터 확보가 어려움
- 전이 학습, 파인튜닝에도 데이터 부족이 한계
- 새로운 ISA 추가 시 재학습 비용 증가

기존 LLM 기반 분석

LLM4Decompile · SymLM

한계 - 단일 함수의 로컬 컨텍스트 중심

- 함수 간 상호작용의 상위 의미 포착이 어려움
- 다수 보조 함수가 결합해 하나의 기능을 구현
- 개방형 함수명 생성 시 환각 이슈 발생 가능

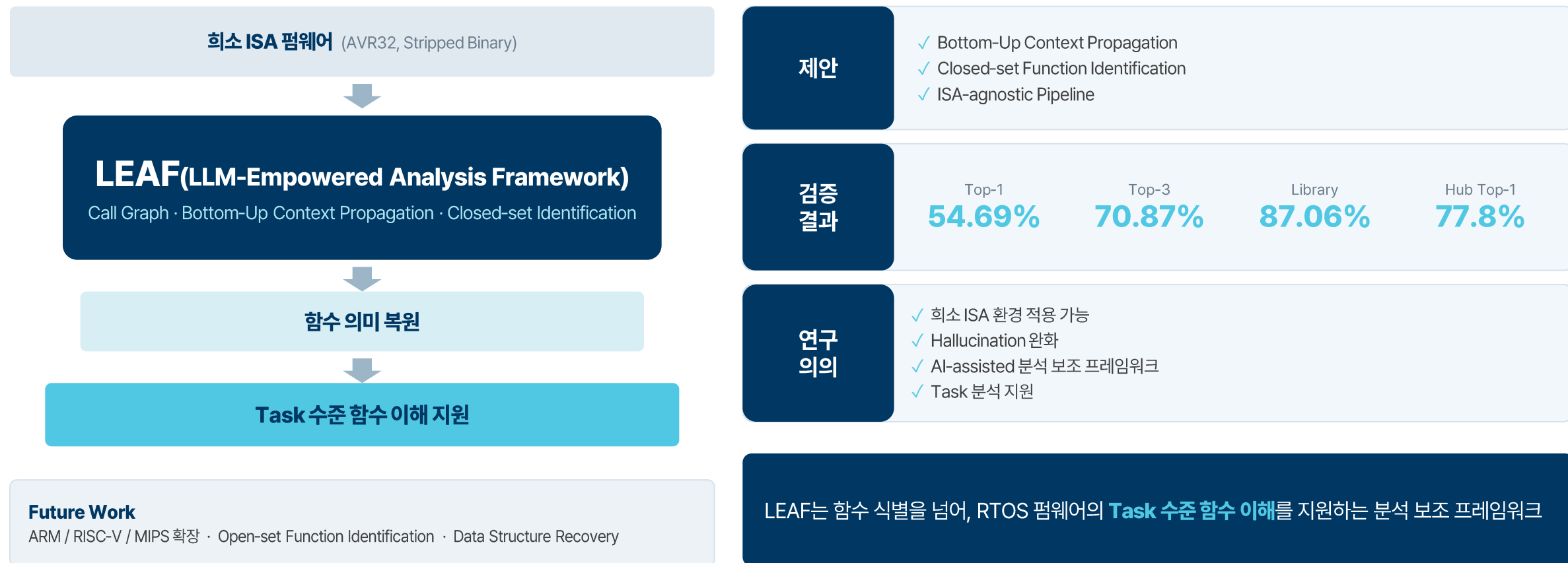
회소 아키텍처 환경에서는 참조 바이너리, 학습 데이터, 단일 함수 문맥에 의존하는 기존 접근만으로 **안정적인 Task, 함수 의미 복원이 어렵다.**

✓ 기존 기법 대비 LEAF의 차별성

Category	Input	Output	Prerequisites	Supported ISA	Hallucination Mitigation
Signature-based	Byte patterns	Function identification	Signature database	ISA with available DB	N/A
Graph Diffing	CFG / graph structures	Function matching	Reference unstripped binary	Multiple ISAs	N/A
Neural-based	Assembly / IR	Function names	Large-scale training data	Primarily x86 / ARM	N/A
LLM-based	Decompiled code	Code / function names	Pretrained LLM	Primarily x86	✗ No
LEAF (Proposed)	Decompiled code + call graph	Closed-set function names	Candidate function list	ISA-agnostic	✓ Yes (Closed-set)

LEAF: Zero-shot 추론 · Closed-set 후보 선택 · Human-in-the-loop 분석 보조

✓ LEAF: 위성·임베디드 펌웨어 분석을 보조하는 LLM 기반 함수 의미 복원 프레임워크



✓ Pros

- **Architecture-portable 설계:** Ghidra 의사코드와 호출 그래프를 추상화 계층으로 사용하여 ISA-agnostic 분석을 지원
- **Hallucination Mitigation:** Closed-set 후보군 기반 추론으로 존재하지 않는 함수명 생성을 억제
- **Human-in-the-loop 적합성:** Top-3 후보·신뢰도·근거를 제시하여 분석가의 검토와 최종 판단을 지원
- **Human-like Reversing Workflow:** 라이브러리·SDK 후보를 추정한 뒤 함수 의미를 좁혀가는 실제 리버싱 과정과 유사

✓ Cons

- **Decompiler Dependency:** Ghidra 의사코드를 입력으로 사용하므로 디컴파일 품질에 성능이 영향을 받음
- **단일 펌웨어 실험:** 단일 AVR32 펌웨어 중심 실험으로, 다양한 펌웨어에 대한 일반화에는 추가 검증 필요
- **Closed-set Assumption:** 후보 함수군 구성에 분석가의 사전 판단이 일부 필요
- **추가 정량 비교 필요:** call graph 제거(ablation) 등 구성 요소별 기여도에 대한 정량 분석은 향후 과제

본 연구는 완전 자동 함수명 확정이 아니라, 회소 ISA 펌웨어에서 초기 분석 비용을 줄이는 **AI-assisted 분석 보조 프레임워크**의 가능성을 검증한 PoC임

Q&A

임베디드 펌웨어 정적 분석 기반 통신 프로토콜 및 Task 식별 방법론 연구

발표자: 연동현 | dhfz1214@gmail.com | 경희대학교 PWNLAB | Theori Korea

